

LeetCode 340

Longest Substring with At Most K Distinct Characters

Comparison: LC904 vs LC340

Feature	LC904 (Fruit Into Baskets)	LC340
Data Type	Integers	Characters
Condition	At most 2 distinct	At most K distinct
Data Structure	HashMap	HashMap
Algorithm	Same algorithm	Same algorithm

Literally only one number changes.

Approach 0 — Brute Force

Intuition

- Generate every substring.
- Maintain character frequencies.
- As soon as distinct characters become greater than k, stop extending.

Code

```
class Solution {
public:
    int lengthOfLongestSubstringKDistinct(string s, int k) {

        int n = s.size();
        int ans = 0;

        for (int i = 0; i < n; i++)
        {
            unordered_map<char,int> freq;

            for (int j = i; j < n; j++)
            {
                freq[s[j]]++;

                // More than k distinct characters
                if (freq.size() > k)
                    break;

                ans = max(ans, j - i + 1);
            }
        }

        return ans;
    }
};
```

Complexity

- **Time:** $O(N^2)$
- **Space:** $O(K)$ (More precisely, $O(\min(K, \text{alphabet size}))$.)

Observation: Exactly the same optimization as LC904. Instead of restarting from every index, keep one moving window.

Approach 1 — Sliding Window (Standard Template)

Intuition

- Maintain frequency of every character.
- If distinct characters become greater than k , shrink.

Algorithm

1. Include current character.
2. If new character enters, frequency map grows.
3. Shrink while distinct characters exceed k .
4. Update answer.

Code

```
class Solution {
public:
    int lengthOfLongestSubstringKDistinct(string s, int k) {

        unordered_map<char,int> freq;

        int l = 0;
        int r = 0;

        int ans = 0;

        while (r < s.size())
        {
            // Include current character
            freq[s[r]]++;

            // Window becomes invalid
            while (freq.size() > k)
            {
                // Remove leftmost character
                freq[s[l]]--;

                if (freq[s[l]] == 0)
                    freq.erase(s[l]);

                l++;
            }

            // Window is valid
            ans = max(ans, r - l + 1);

            r++;
        }

        return ans;
    }
};
```